University of Cape Town
Department of Computer Science

Computer Science CSC3003S
Class Test 2, 2025

Answer all questions.
Marks per question are shown in brackets.

Time: 45 minutes (+5 minutes reading time) **Marks: 35**

Section 1: Theory of Algorithms [20]

Question 1. **[4]**

Select all options that apply in the following questions.

a) if $T(n) = n^2 + (\log n)^2$, then T(n) is in

i. $O(n^3)$
ii. $\Theta(n^2)$
iii. both i. and ii.
iv. none of the above

ANS: iii (1 mark) or i (0.5 mark and ii 0.5 marks)

b) Is the following statement True or False?

"We can use the Master Theorem to solve a recurrence of the form $T(n) = aT(n-b) + f(n)$, where f(n) is in $\Theta(n^d)$"

ANS: False (1 mark)

c) Given $T(n) = T(n/3) + 3$, the solution to the recurrence is in

i. $\Theta(n^2)$
ii. Θ (log n)
iii. Θ (n)
iv. Θ (1)

ANS: ii (1 mark) simple application of Master Theorem at end of Qn paper)

d) Solving the Travelling Salesman (TSP) problem, between *n* cities, by brute force enumeration (exhaustive search) is in

i. Θ (2^n)
ii. Θ (n^n)
iii. Θ (n!)
iv. none of the above

ANS: iii (1 mark) (reason: ~ n! paths through all cities – see notes)

Question 2. **[3]**

You are given an unsorted array of n elements and asked to find the kth smallest element.

a) Explain briefly how you could do this *without* first sorting the array. [2]
b) Why will this approach not always be better than presorting? [1]

ANS:

a) recursively apply a partitioning scheme like Hoare or Lomuto with a suitable pivot until you reach a state where we have k-1 elements to left of pivot and remainder on right. The pivot is then the kth smallest in array

(**1 mark** for recursive partitioning about pivot),

(1 mark for final partition with pivot in the kth position – note it is also fine - for 0-indexed arrays - to say the pivot must be in (k-1)th (array index) position)

The details of the recursive algorithm – quickselect – are not needed.

Also, just mentioning quickselect is only worth **0.5 mark**.

b) for some arrays contents and pivot strategies this can degenerate to $\Theta/O(n^2)$ comparisons – so sort O(n log n) would be better for these array types. (**1 mark** for noting it can degenerate to $\Theta/O(n^2)$)

Question 3. **[2]**

Why does *interpolation search* not always reduce the search domain by a constant factor like binary search?

ANS: Interpolation search estimates the key index position by linearly interpolating the *key values* of current search array end points - this can lead to a split where the new left and right sub-arrays are of different sizes **(1 mark – emphasize the interpolation causes the uneven split)**

This is different from binary search which always estimates the key position as the middle of the previous search interval. **(1 mark)**

Question 4. **[4]**

You are given an array of elements $[s_1, ..., s_n]$ and asked to write an algorithm to sum all the elements. Your classmate claims you can improve on brute force summation (n-1 adds) by using a simple *divide and conquer* algorithm to do the summation in "log n operations"! Show that this is not true by defining and solving the recurrence relation for the number of *additions* A(n) required to compute the full sum using the obvious divide-and-conquer approach for n elements. After you have defined the full recurrence relation, you can assume that $n = 2^k$ to solve it – you may use the Master Theorem. NOTE: this is a *serial* solution, so no parallel sums.

ANS: Split array into two halves of size floor(n/2), ceiling(n/2) **(1 mark)**
(note – just saying n/2 is fine for **0.5 mark**)

Recurrence: A(n) = A(floor(n/2)) + A(ceil(n/2)) + 1
(**1 mark** for full recursive formulae – **or 0.5 mark** if floor(.)/ceil(.) not used))
A(1) = 0 **(0.5 marks)**
if $n = 2^k$:
We can use back-substitution (which is long – so not advised) or just use Master Theorem
A(n) = 2 A(n/2) + 1 **(0. 5 mark)**
a = 2, b = 2, d = 0
- ⇨ 2 > 2^0 =1
- ⇨ Θ(n^log_b a) = Θ (n^1) = Θ (n)) **(1 mark)**
- ⇨ (which is not Θ /O(log n)….)

Question 5. **[4]**

Use Horner's algorithm for exponentiation to compute the number of multiplies needed to compute a^{13}. **Show all your work**.

ANS:

13 = 8+4+1 = 1101 in binary **(1 mark)**

p = a

bit	1	1	0	1	DONE!
	P=a	P ← (P*P)*a	P ←P*P	P ←P * P*a	
Num mults		2	1	2	

1 mark for showing #multiplications in **each** highlighted columns above. **= 3 marks**

(Just stating 5 multiplications get 1 mark only.)

Question 6. **[3]**

You are given the following solution table for the knapsack problem of capacity W = 6 and n=5 items.

		capacity j						
	i	0	1	2	3	4	5	6
	0	0	0	0	0	0	0	0
$w_1 = 3, v_1 = 25$	1	0	0	0	25	25	25	25
$w_2 = 2, v_2 = 20$	2	0	0	20	25	25	45	45
$w_3 = 1, v_3 = 15$	3	0	15	20	35	40	45	60
$w_4 = 4, v_4 = 40$	4	0	15	20	35	40	55	60
$w_5 = 5, v_5 = 50$	5	0	15	20	35	40	55	65

Use this information determine the most valuable items that can fit into a knapsack of capacity W'= **5**. Show your reasoning – based on table values - used to determine the solution set – guess will not be accepted.
ANS:

NOTE: The table contains *all* sub-problems for W < 6 and up to 4 items – so we just backtrack from the appropriate position.

Start at i = 5, j = 5 .
T(5,5) = T(4,5) -> so item 5 not in knapsack **0.5 mark**
Continue from T(4,5) **0.5 mark**
T(4,5) > T(3,5) -> so **item 4** is in knapsack. **0.5 mark**
Now, move w_4 (=4) steps left from T(3,5) to get to next start trace position
Continue from T(3,1) **0.5 mark**
T(3,1) > T(2,1) **-> item 3** is in knapsack! **0.5 mark**
Now, move w_3 (=1) left from T(2,1) = 0 -> terminate. **0.5 mark**

So knapsack contains {item 3, item4}

(if nothing else is shown, but its observed that we startsat T(5,5) that can be awarded 0.5 marks)

[NB: MUST SHOW WORKING – by inspection the answer is clear, so that is not enough, the backtracing process must be followed or clear from the presented answer.]

Section 2: Theory of Computation [15]

Question 7. **[7]**

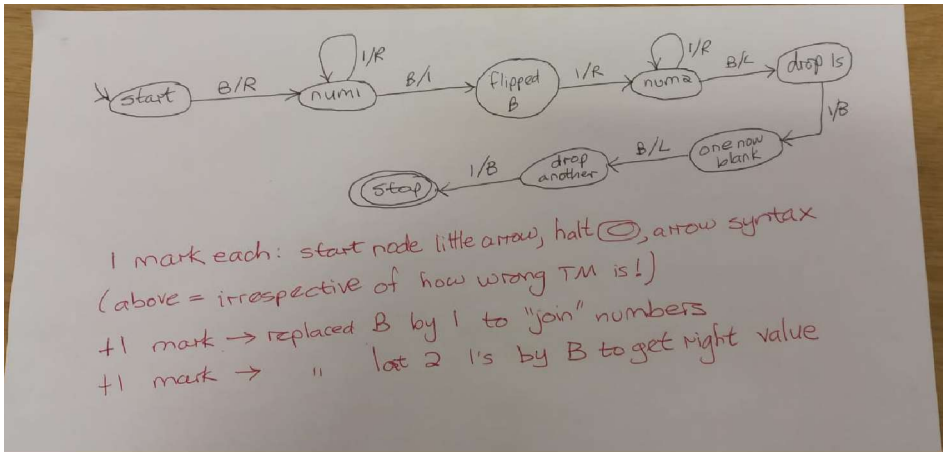
a) Using B to represent a blank cell, **draw a transition diagram for a Turing Machine** with alphabet {B,1} that does natural number addition for unary numbers, and stops with the tape head at the cell immediately right of the total. A number N is represented by a string of N+1 consecutive 1s. The tape head will start on a blank cell, followed by the two numbers, with a single blank cell between them. Example input for 3 + 1:

```
B1111B11BB- - -
↑
```

which must generate output:

```
B11111BBBB- - -
     ↑
```

[5]



b) **How many rows** would your Turing Machine above have in its State Table? [1]

As many rows as there are arrows in THEIR diagram (whether diagram is right or wrong, if answer here is number of arrows it has, then they know it's a row per arrow)

c) Give an example of **any one row** in your Turing Machine's State Table. [1]

Again, even if their TM is wrong, mark this based on THEIR machine. Answer must contain 4 things for any one arrow in their diagram; mark right no matter what order the 4 things are given in. Don't miss the arrow they are using!! No half marks: 0 or 1

Current state: state where arrow starts,
Current/tape symbol: symbol before the /,
Next state: state where arrow ends,
Action: symbol after the /

Question 8. **[3]**

Give a regular expression (**regex**) that accepts the language AskLess:

AskLess = { w | w comprises 2 binary numbers separated by either "<" or "<=" }

Example strings in this language: 10101<=11011 or 11011<=10101 etc. – i.e. a language for stating that kind of Boolean expression.

(1|0)+<=?(1|0)+ or (1|0)+[<|<=](1|0)+ or (1|0)+<[=| ε](1|0)+

i.e. always (1|0)+ on each end, and the comparator in between can be represented as <=?
[< | <=]
< [= | ε]
they could also (optionally) put explicit concatenate dots between things
One mark or zero, no half marks

Question 9. **[1]**

We know there are languages no Turing Machine can recognize because (A) the set of all languages is *uncountably* infinite; and (B) the set of all Turing Machines is *countably* infinite. **In one sentence**, **explain** briefly how we know (B) is true.

Because everyTM can be represented by a binary number [½]
And binary numbers are countable [½]

Question 10. **[4]**

Which of the following statements are **false**? **Give the letter(s) of the false** statement(s), with a brief reason for your answer. **Negative marking employed**: right minus wrong! Minimum mark zero.

a) If the input of a problem is of size N, then the lower bound of that problem must be at least Ω(N).
b) Using the adversary argument in a "Guess the 2-digit number" game, one can keep changing the correct answer after each guess. For example, start with 48, then make it 62, then 55, etc.
c) Any language that is decidable is also acceptable.
d) If problem A has a lower bound of N log N, and we can show that problem B is reducible to problem A in linear time, then problem B also has a lower bound of N log N.

ANSWER:

(a) is false, e.g. binary search
(b) is false, as 2nd value must be <48, because fewer 2-digit numbers below 48 than above it
(d) is false, we'd need to show A is reducible to B, i.e. the other way round is correct

MARKING:

(a) is false [1] (no marks for reason)
(b) is false because 62 is wrong [1] **or (b) is false because changed values are wrong [1]**
or (b) is false because changed values too big [1]
or (b) is false because changed values not < 48 [1]
i.e. one mark for knowing why it is false, else no marks
(d) is false [1] (no marks for reason)
[1] for NOT listing (c) (know it's true)
or MINUS ½ FROM ABOVE TOTAL if they say (c) is false
Minimum zero

Appendix A – Master Theorem

If we have a recurrence of this form: $T(n) = aT(n/b) + f(n)$, $f(n) \in \Theta(n^d)$ and $T(1) = c$ then

$T(n) \in \Theta(n^d)$ if $a < b^d$

$T(n) \in \Theta(n^d \log n)$ if $a = b^d$

$T(n) \in \Theta(n^{\log_b a})$ if $a > b^d$